

INSuRE Spring 2023 Midterm Presentation

Malware Similarity

Sponsor: MITRE

Technical Director
Frank Posluszny(malwarefrank@mitre.org)

Faculty
Brian Ricks(bwr031000@utdallas.edu)



Team

Kushan Singh (kxs200085@utdallas.edu)

Computer Science, UTD

Haoyan Li (hxl210024@utdallas.edu)

Computer Science, UTD

Jerry Teng (jerry.teng@utdallas.edu)

Computer Science, UTD

Payton Harmon (pnh170000@utdallas.edu)

Computer Science, UTD

Surekha Kumari (sxx210055@utdallas.edu)

Computer Engineering, UTD



Motivation

- Detection and prevention of malware attacks.
- Identifying source and origin of malwares.
- Understanding malware evolution.



Goals

Studying string similarities in malware analysis:

- Identifying common malware families.
- Analyzing malware variants.
- Understanding malware behavior.



Literature Review



Figure 1 : Malware Development Life Cycle (simplified)

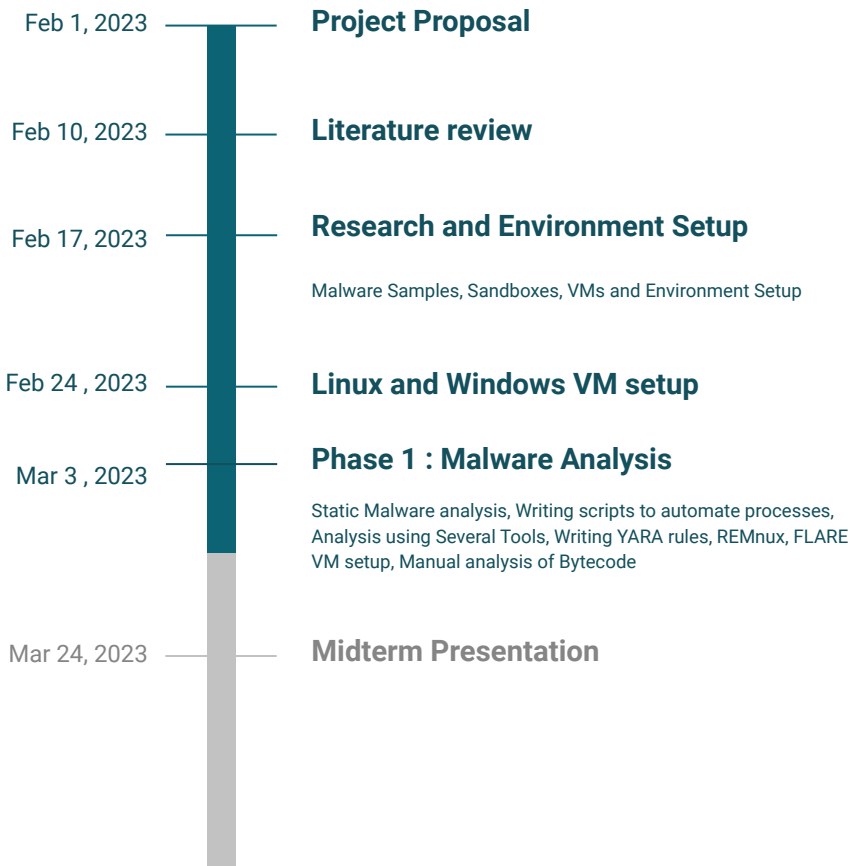
Malware Similarity Analysis: An adversary perspective - Frank Postlusny, March 2020

Software Forensics: Can We Track Code to its Authors? - Eugene H. Spafford, Stephen A. Weeber, February 1992

Automated Attacker Correlation for Malicious Code - Thomas Dullien, Ero Carrera, Soeren-Meyer Eppler, Sebastian Porst, March 2010



Timeline





Challenges

- Despite a large amount of available data, there is currently a need for deeper analysis to derive meaningful insights
- Project Environment Setup
 - Windows and Linux Virtual environment support on Windows and M1 Mac Machines
- Limited and varying domain specific knowledge among team members
- Difficulty in finding malware samples that accept command line arguments



Current Malware Samples

- Analyzing binaries for tools commonly used by malicious actors
 - Accept command line arguments
 - Provided by technical director

Name	Functionality
AdFind	Query Active Directory
NirSoft	Extract passwords saved in browser
Onhat	Network proxy
Ransomware	assorted ransomwares
RemCom	Remote Administration Tool
Upchurch Group 2	Network proxy
X-tunnel	Network proxy



Progress so far

- Setting up virtual machines for static and dynamic analysis
 - REMnux, FLARE VM
- Information extraction and aggregation
 - Run tools to extract information from the binary, including strings, libraries, and capabilities
- Manually analyzing bytecode to see patterns in usage strings/CLI arguments
 - Current detection methods only look at Windows API functions (e.g. GetCommandLine, CommandLineToArgv)
 - Looking for bytecode heuristics to determine whether a program accepts/uses command line arguments
 - Discovered potential bug in Ghidra and FLOSS/vivisect frameworks involving stackstrings

Ghidra:

Hybrid Analysis (online sandbox):

```
@403b16: lea eax, dword ptr [esp+00000148h] ;ONTAH.EXE [-f N] LOCATION REMOTION [-b ADDR] [-a ADDR] [-p]-P PORT]
```

```
320     local_178[0] = 'O';
321     local_178[1] = 'N';
322     local_178[2] = 'T';
323     local_178[3] = 'A';
324     local_178[4] = 'H';
325     local_178[5] = '.';
326     local_178[6] = 'E';
327     local_178[7] = 'X';
328     local_178[8] = 'E';
329     local_178[9] = ' ';
330     local_178[10] = '[';
331     local_178[11] = '-';
332     local_178[12] = 'f';
333     FUN_004046d0((char)local_178);
```



AdFind

```
PS C:\Users\flare\Desktop\Malware Samples\INSURE 2023-01 UTD Team 4\adfind-xfer\adfind> .\AdFind.exe -help
```

```
AdFind V01.57.00cpp Joe Richards (support@joeware.net) November 2021
```

```
-help      Basic help.
?          Basic help.
??         Advanced/Expert help.
????      Shortcut help.
-sc?       Shortcut help.
-meta?     Metadata help.
-regex?    Regular Expressions help.
-gui       Combine with help switch to open that output in text editor.
```

Usage:

```
AdFind [switches] [-b basedn] [-f filter] [attr list]
```

```
basedn      RFC 2253 DN to base search from.
             If no base specified, defaults to default NC.
             Base DN can also be specified as a SID, GUID, or IID.
filter      RFC 2254 LDAP filter.
             If no filter specified, defaults to objectclass=*.
attr list   List of specific attributes to return, if nothing specified
             returns 'default' attributes, aka * set.
```

Switches: (designated by - or /)

[CONNECTION OPTIONS]

```
-h host:port Host and port to use. If not specified uses port 389 on
             default LDAP server. Localhost can be specified as '.'.
             Port can also be specified via -p and -gc.
-gc          IPv6 IP address w/ port is specified [address]:port
             Search Global Catalog (port 3268).
-p port      Alternate method to specify port to connect to.
```

[QUERY OPTIONS]

```
-s scope     Scope of search. Base, One[Level], Sub[tree].
-t xxx       Timeout value for query, default 120 seconds.
```

[OUTPUT OPTIONS]

```
-c           Object count only.
-dn          Object DN's only.
-appver      Output AdFind versioning info.
```

Notes:

- o This tool was written with simple US ASCII in mind. UNICODE and special ASCII characters such as characters with umlauts or graphics may not be output correctly due to how the command prompt handles those characters. If you see this occurring, redirect the output to a text file with the command prompt redirection symbol (>) and it is possible the program will give the desired output.

Ex1:

```
adfind -b dc=joehome,dc=net -f "objectcategory=computer"
Find all computer objects in joehome.net and displays all attributes
```

Ex2:

```
adfind -b dc=joehome,dc=net -f "objectcategory=computer" cn createTimeStamp
Find all computer objects in joehome.net and displays cn and createTimeStamp
```

Ex3:

```
adfind -h .:50000 -b cn=ab -f "objectcategory=person"
Find all person objects on cn=ab container of local ADAM instance
```

This software is Freeware. Use at your own risk.

I do not warrant this software to be fit for any purpose or use and I do not guarantee that it will not damage or destroy your system. Contact support@joeware.net via email for licensing information to package this utility in commercial products.

See full Warranty documentation or download the latest version on <http://www.joeware.net>.

If you have improvement ideas, bugs, or just wish to say Hi, I receive email 24x7 and read it in a semi-regular timeframe. You can usually find me at support@joeware.net



Onhat

```
PS C:\Users\flare\Desktop\Malware Samples\INSuRE 2023-01 UTD Team 4\onhat-xfer\onhat\onhat> .\02f1de3e80054bbbe8579aa3bbba81f18634d27e05d3a41c2a9e4448cc6187f7.exe -h
INVALID FARPROC. TYPE ONTAH.EXE -h FOR HELP INFORMATION.
PS C:\Users\flare\Desktop\Malware Samples\INSuRE 2023-01 UTD Team 4\onhat-xfer\onhat\onhat> _
```

[Sandbox](#)[Quick Scans](#)[File Collections](#)[Resources](#)[Request Info](#)

No relevant HTTP requests were made.

Extracted Strings

[Search](#)[All Details: Off](#)[Download All Memory Strings \(463B\)](#)[All Strings \(93\)](#)[Interesting \(45\)](#)[02f1de3e80054bbbe8579...](#)[screen_O.png \(7\)](#)[file.exe:3288 \(24\)](#)

H_ENV_SERV_BIND

INVALID FARPROC. TYPE ONTAH.EXE -h FOR HELP INFORMATION.

INVALID PARAMETERS. TYPE ONHAT.EXE -h FOR HELP INFORMATION.

ONTAH.EXE [-f N] LOCATION REMOTION [-b ADDR] [-a ADDR] [-p] -P PORT

ONTAH_ENb:ef:hlo:O:p:P:s:t

ONTAH_ENV_SERV_BIND

ONTAH_ENV_SERV_PORT

ONTAH_ENV_THRU_BIND

ONTAH_ENV_THRU_PORT_X

ProviderInfo

RV_PORT

StoresServiceClassInfo

Custom script (run_tools.sh)

```
#!/bin/bash

while getopts "d:" arg; do
    case $arg in
        d) DIR=$OPTARG;;
        esac
    done

    echo Scanning directory $DIR
    files=$(find $DIR \
        -type f \
        ! -name "*.zip" \
        ! -name "*.floss" \
        ! -name "*.capa" \
        ! -name "*.strings" \
        ! -name "*.objdump" \
        ! -name "*.txt" \
        ! -name "*.pefeat" \
        ! -name "*.err" \
        ! -name "*.chm" \
    )

    for file in "${files[@]}; do
        python3 detect_packed.py $file 1>/dev/null
        is_packed=$(echo $?)
        if [[ $is_packed == 2 ]]
        then
            continue
        fi

        echo Processing $file
        echo -e "\t" Running PEFeatExt
        python3 PEFeatExt/PEFeatExt.py $file > "$file".pefeat 2> "$file".pefeat.err
        echo -e "\t" Running FLOSS
        floss $file > "$file".floss 2> "$file".floss.err
        echo -e "\t" Running capa
        capa -v $file > "$file".capa 2> "$file".capa.err
    done

    # done | pv -s "${#files[@]}"
```

```
remnux@remnux ~/code/malware_similarity$ ./run_tools.sh -d ~/samples/cli-malware/onhat/onhat/f8dc94ad67b49fd6a8f5755a4acced0f585321e78122c2084872e6605a533f61
Scanning directory /home/remnux/samples/cli-malware/onhat/onhat/f8dc94ad67b49fd6a8f5755a4acced0f585321e78122c2084872e6605a533f61
Processing /home/remnux/samples/cli-malware/onhat/onhat/f8dc94ad67b49fd6a8f5755a4acced0f585321e78122c2084872e6605a533f61
Running PEFeatExt: /home/remnux/samples/cli-malware/onhat/onhat/f8dc94ad67b49fd6a8f5755a4acced0f585321e78122c2084872e6605a533f61.pefeat
Running FLOSS: /home/remnux/samples/cli-malware/onhat/onhat/f8dc94ad67b49fd6a8f5755a4acced0f585321e78122c2084872e6605a533f61.floss
Running capa: /home/remnux/samples/cli-malware/onhat/onhat/f8dc94ad67b49fd6a8f5755a4acced0f585321e78122c2084872e6605a533f61.capa
Running objdump: /home/remnux/samples/cli-malware/onhat/onhat/f8dc94ad67b49fd6a8f5755a4acced0f585321e78122c2084872e6605a533f61.objdump
```



Next Steps

Long Term Goal: Establish a reliable method of determining whether a malicious binary takes command line input.

Short Term Goal 1: Develop a script that utilizes existing tools to collect artifacts from known binaries.

Short Term Goal 2: Analyze known binaries using byte code analysis to look for patterns that can be detected using capa or Yara.



Any Questions?